Swissquote Wallet — Your key. A way back.

Track: Swissquote — Build the Swissquote Wallet
Category: Self-custody wallet, recovery and sponsored payments
Network: Ethereum Sepolia; isolated local development network also supported
Submission: Hackathon prototype; source, technical pitch and evidence included
Team members: Mehrshad Mirmohammadi; Erfan Mousavian

One-line summary

A self-custody wallet with gas-sponsored payments, card-plus-helper recovery and an independent Emergency Kit, built on Safe and demonstrated on Sepolia.

The problem

Self-custody gives clients control but makes a lost or compromised key frightening. A bank-like user experience should not require giving the bank ordinary control of the wallet. Clients need simple payments, understandable recovery and an exit if a service disappears.

What we built

Our web wallet uses a one-owner Safe. The client's device creates the owner key locally and stores it encrypted. A separate Recovery Desk creates the Recovery Card, checks wallet configuration and opens Emergency Kits. Two separately running helper services represent Swissquote and a biometric partner. Swissquote sponsors approved transactions without receiving the client's ordinary payment key.

The client journey is create wallet → verify configuration and save backup → receive test USDC → sign and send → recover access on a replacement device → pay again from the same wallet address. We built a guided recovery interface, a visible waiting period, cancellation from the existing owner, transaction previews and an explanation of who can move funds.

Custody and threat model

The normal payment authority is the single Safe owner: the client's key. Login authenticates the client to the service; it does not itself sign a payment or decrypt the device key.

Recovery uses two independent module instances. Module A requires the Recovery Card plus the partner and has a three-minute demonstration delay. Module B requires the Recovery Card plus Swissquote and has a three-day delay. Each module requires both of its configured guardians. Swissquote alone, the partner alone, or both helpers without the card cannot satisfy either intended threshold. The current owner can cancel pending recovery.

The Recovery Card is not the ordinary spending key. The Emergency Kit is different: its encrypted file plus separate kit code can recover the owner seed. Those materials must be stored separately. An old kit no longer controls the wallet after the owner has been replaced.

This design does not eliminate all threats. A stolen unlocked owner key can spend or alter the wallet. A malicious or compromised publisher can serve JavaScript that steals secrets. A stolen card plus a fooled matching helper can initiate recovery. Cancellation depends on detection, availability and transaction ordering. Identity checks and helper-key vault protections remain mocked. This is not an audited production system.

Architecture — real versus mocked

Component	Implemented behavior	Prototype boundary
Wallet app	React/TypeScript; local key generation and encryption; Safe transaction signing; previews; guided recovery	Browser delivery and local storage are not hardened production key custody
Safe wallet	Safe v1.4.1, one owner, deterministic setup, public chain execution	Sepolia testnet, not production assets
Recovery modules	Candide Social Recovery Module v0.0.1; distinct card-required paths; delayed owner replacement	Three-minute partner delay is for demonstration; three-day Swissquote completion has not been shown live
Recovery Desk	Separate browser origin; card generation, independent public-node checks and kit opening	Same project publisher; not an independently audited distribution
Swissquote service	Login, ownership registration, approved transaction relay and fee sponsorship	Bank balance/origin, KYC and helper-key vault protection are mocked
Partner service	Separate process/store; per-guardian approval path	Face/video assurance is mocked, not real biometric proof
Screening	Public-chain sanctions-oracle and USDC blacklist reads; sponsorship checks	Demo entries and some coverage are mocked or incomplete; no claim of regulatory clearance

The included architecture image shows these boundaries. The source contains the wallet, Desk, helper services, protocol code, contract fixtures and tests.

Wallet creation and identity

The wallet generates a 24-word BIP-39 owner seed, derives the Ethereum key at m/44'/60'/0'/0/0, and encrypts the seed using PBKDF2-HMAC-SHA-256 with 600,000 iterations and AES-GCM. The encrypted record is stored in IndexedDB. The device passphrase is separate from the account login password.

The separate Desk generates a 12-word Recovery Card. Only its public address is transferred to the wallet; card words are not requested by the wallet app. The wallet constructs a deterministic Safe configuration and uses public-node checks to compare the expected address, owner, modules, guardians and thresholds. A purpose-limited Safe signature proves control when registering the wallet against the authenticated account. Account identity verification itself remains MOCK.

Authentication and payments

Clients can sign in using password or an enrolled passkey. The owner key signs the Safe payment. Swissquote submits the signed transaction and pays the network fee when sponsorship policy permits, so the normal test-USDC payment can work with no ETH in the owner or wallet.

The normal UI uses USDC. Payments above 5 USDC require hash-bound passkey step-up for sponsorship; password login is not a substitute for that step-up. Device-key unlocking remains separate. Sponsorship policy is not an on-chain spending limit: an owner can self-fund or use another submitter.

Recovery, theft and service disappearance

On a replacement device the client prepares a new encrypted owner key, reviews the replacement address, obtains a card approval through the Desk and a matching helper approval, and submits recovery. After the waiting period the owner can be replaced without changing the wallet address. The old owner's signature is then invalid. Pending requests on both modules must be considered; the UI detects a conflicting other-module request but requires manual handling.

If Swissquote is unavailable, the card-plus-partner path does not require Swissquote to be a guardian. If helper services are unavailable, the Emergency Kit provides the original owner material and instructions for using a funded independent submitter. The retained exit demonstration used an RPC-only script; external MetaMask-to-Safe spending and an actual all-services-down rehearsal have not been demonstrated. Working chain access and transaction fees are still required.

For a stolen unlocked owner key, recovery is not a refund or reversal mechanism. Funds already stolen cannot be restored by changing the owner. The included threat-model discussion distinguishes losing access from leaking a spending key.

Public execution evidence

These receipts are from retained earlier testnet runs, not newly generated for this submission archive.

- Sponsored payment, with no owner/wallet ETH required: https://sepolia.etherscan.io/tx/0xe3188195628404c7da0883744aa6562c45e6597fe32c636e72fb3890801dc77e
- Partner recovery finalized after a real waiting period: https://sepolia.etherscan.io/tx/0x2de7323c1e8de99119cd2ba651794e90e252c3c0729961bd8d7132485c62fa5e
- Replacement owner paid from the same wallet: https://sepolia.etherscan.io/tx/0xaf43a7e72b8f8f951fef5d68b73700525416e1581f25c44e81edc2de3053976a
- Separate built-in DeFi supply: https://sepolia.etherscan.io/tx/0xa44904bd58728a3cdeef571a6c0aa7c23aaa5af7cad7bc618fafd2ea1b3694af

The public browser recovery story completed across resumed encrypted browser sessions after timeouts, not one uninterrupted recording. Separate script evidence covers both-helper refusal without the card, cancellation and an Emergency Kit exit. The active Demo lab explains attack outcomes but does not execute those attacks. The old-key test is a deliberate proof setup rather than a naturally available action on a completely new device.

Bonus work

Gasless experience: real service-sponsored Safe submission is supported by the retained no-ETH payment evidence. This uses a relayer, not an ERC-4337 paymaster claim.

Built-in DeFi: the application has allow-listed actions, previews and screening. A separate browser run supplied test USDC to a third-party Compound III test deployment. This is not an official Compound market, a general WalletConnect integration, an investment recommendation or evidence of a completed withdrawal/swap.

Custody, Travel Rule and legal position

Our technical design supports client control of ordinary spending, but that does not settle Swiss legal classification. Conditional recovery authority, application publication, sponsored execution and the full service need counsel and bank review. We do not claim a FINMA ruling, exemption or production approval.

A proposed bank-to-wallet boundary associates the authenticated client with a wallet through a purpose-limited ownership signature. Signature validity, KYC, beneficial ownership and transaction approval are different checks. A wallet ownership signature alone does not prove a complete Travel Rule process.

For sponsored wallet payments the client signs the exact destination and amount while the relayer submits and pays the fee. That is a technical distinction, not a legal conclusion that Swissquote cannot be an obligated intermediary. Production would require approved AML, sanctions coverage, records, privacy arrangements, liability allocation and incident response. The current screening is a bounded prototype control, not comprehensive Swiss compliance.

Business case

The target customer wants self-custody without losing the familiar bank relationship. Swissquote could offer optional recovery assistance and sponsored payments while maintaining an easy path back to its trading services. The Emergency Kit deliberately permits exit rather than locking the client in.

Revenue hypotheses include an opt-in assistance subscription and retained trading engagement. They are hypotheses, not validated demand or forecasts. For transparent unit arithmetic only, an assumed CHF 4.90 monthly subscription is CHF 58.80 gross per paying account per year; 2,000 assumed subscriptions would be CHF 117,600 gross annually before costs. No actual subscribers, tariff approval, retention lift or profit is claimed.

A next-stage pilot should measure successful setup and recovery completion, comprehension of card versus kit, support burden, fraud/refusal outcomes, sponsorship cost and willingness to pay. Production rollout depends on independent security and legal review.

AI tools

AI assistants accessed through DeepSeek Harness helped with implementation, research synthesis, parallel design/security critique, tests, documentation and presentation. Confirmed tooling included gpt-6.1-sol and Archify. AI is not part of the runtime signing or recovery trust model. Individual contributors' unverified model usage is not invented.

Verification and release boundary

Earlier test and execution records support specific implemented properties. A historical full 609-test gate predates subsequent UI changes; a later full run failed during recovery waiting. The latest prior scoped wallet run reported 54 passing tests and one service-dependent skip, and the audit ran 20 additional mocked checks successfully. Those are not a freshly passing full suite for this exact archive. A clean-machine install and physical passkey ceremony were not repeated for packaging.

The archive contains the current V4 source snapshot, including uncommitted changes. It is the submitted source, not a claim that a public repository branch already matches. Runtime secrets, private environment files, user databases and installed dependencies are excluded. Test fixtures may contain publicly known development keys; never fund them outside isolated development.

Submission contents

The archive includes this submission in Markdown/PDF, the selected technical pitch in HTML/PDF, speaker notes, architecture, retained public evidence, source code, local-run instructions and file checksums. The pitch is viewable offline. Running source requires development dependencies and package downloads. There is no packaged live-demo video and no public hosted demo; this must not be represented as one.

Team members: Mehrshad Mirmohammadi and Erfan Mousavian. Contact details are supplied through the submitting DoraHacks account.